

자료구조

06 주차

연결된 구조



6장. 연결된 구조



6.1 연결된 구조란?

6.2 단순 연결 구조의 응용: 연결된 스택

6.3 단순 연결 구조의 응용: 연결된 리스트

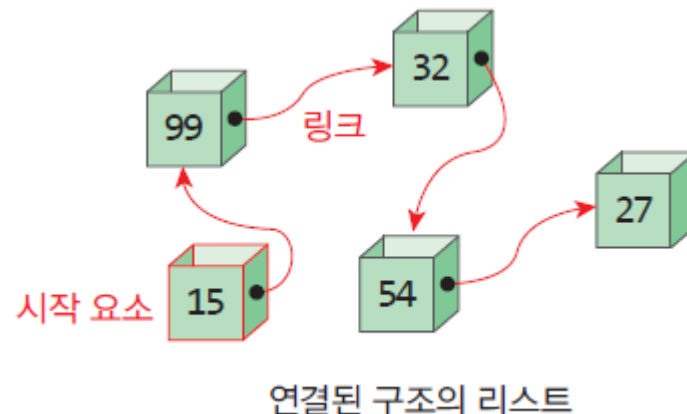
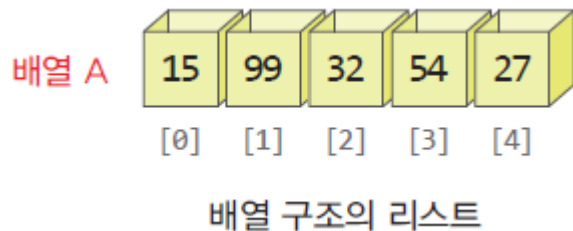
6.4 원형 연결 구조의 응용: 연결된 큐

6.5 이중 연결 구조의 응용: 연결된 덱

6.1 연결된 구조란?

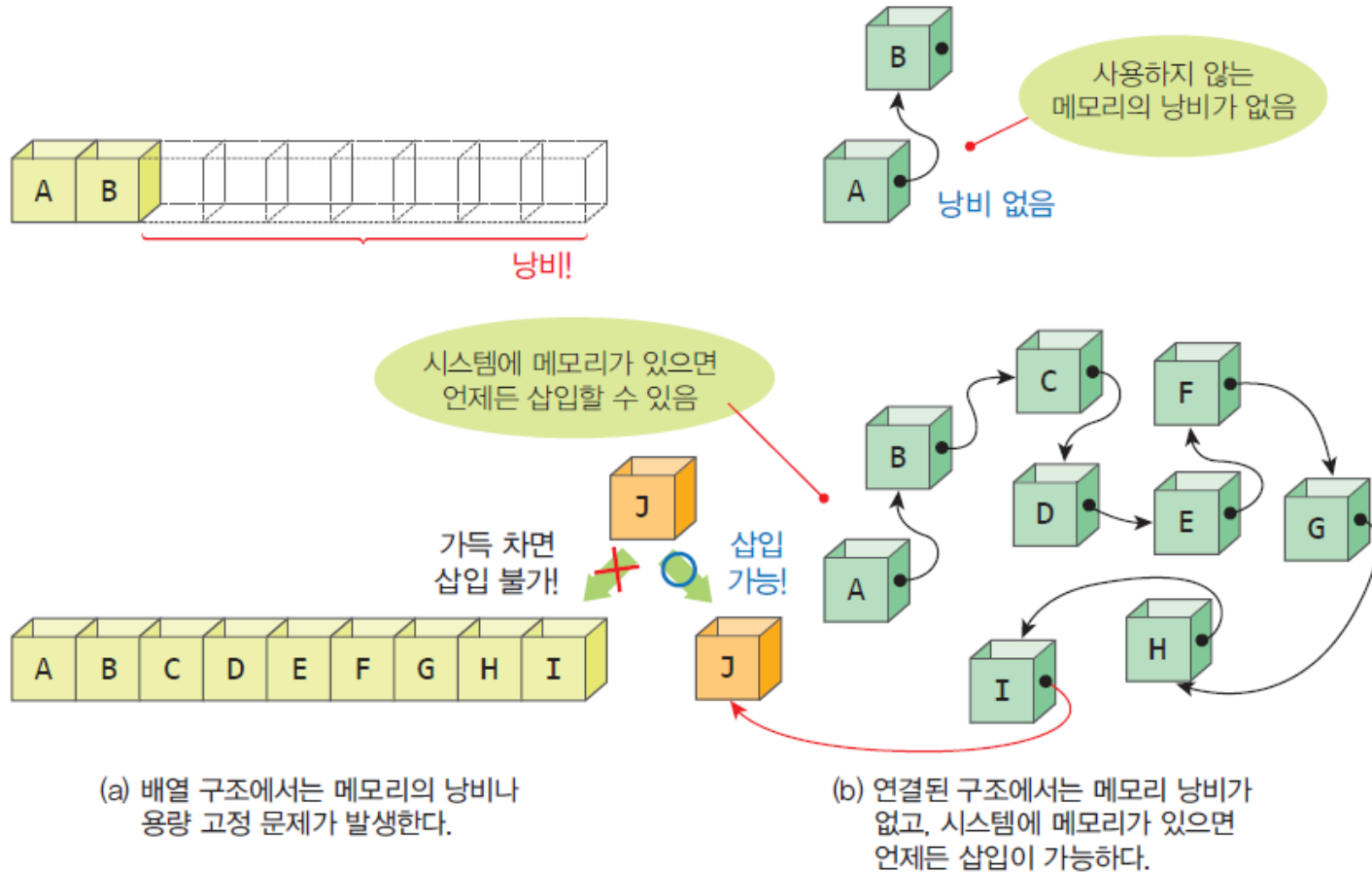


- 배열 구조
 - 모든 데이터를 연속된 메모리 공간에 저장
 - n 번째 자료의 위치를 금방 찾을 수 있음: $O(1)$
- 연결된 구조 (linked structure)
 - 데이터를 한군데 모아두는 것을 포기
 - 흩어진 데이터를 링크로 연결해서 관리
 - n 번째 자료의 위치를 찾으려면? $O(n)$



연결된 구조의 특징

- 메모리 낭비가 없고 용량이 고정되지 않음.

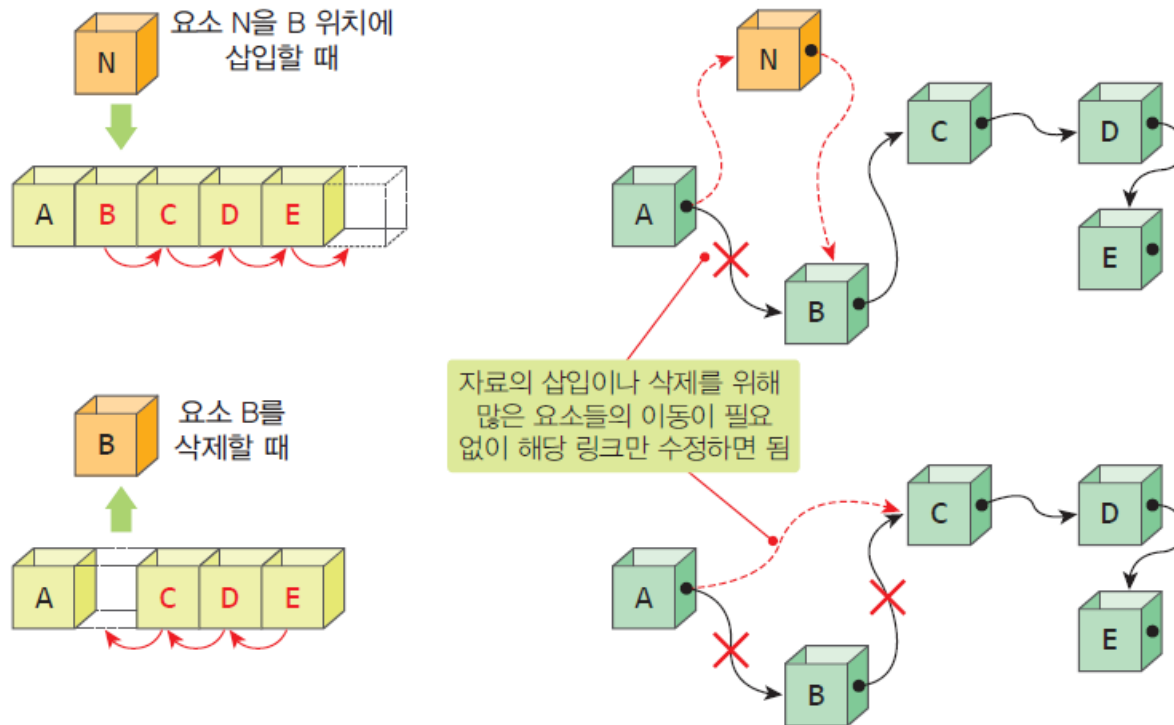


(a) 배열 구조에서는 메모리의 낭비나 용량 고정 문제가 발생한다.

(b) 연결된 구조에서는 메모리 낭비가 없고, 시스템에 메모리가 있으면 언제든지 삽입이 가능하다.



- 중간에 자료를 삽입하거나 삭제하는 것이 용이

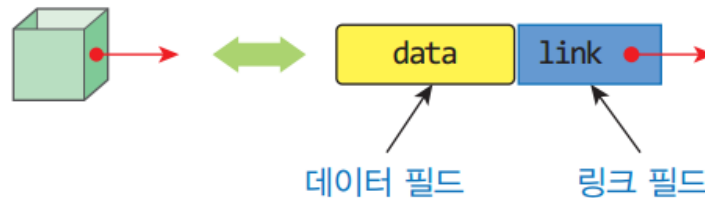


- n 번째 항목 접근에 $O(n)$ 의 시간이 걸림.

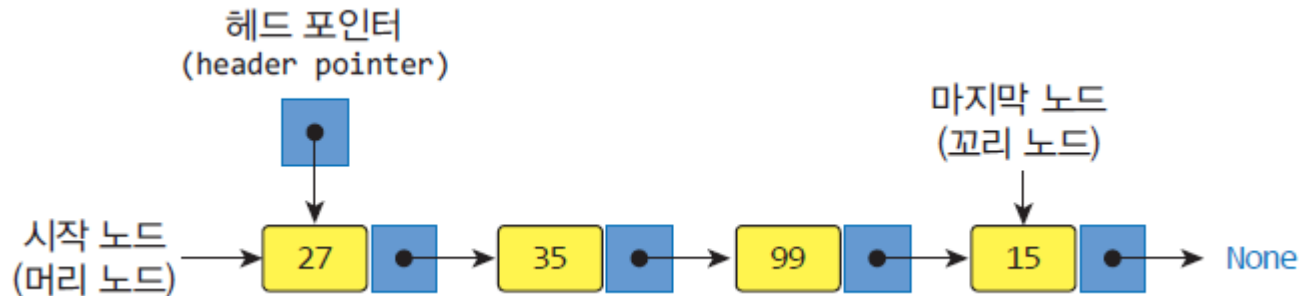
연결된 구조의 용어



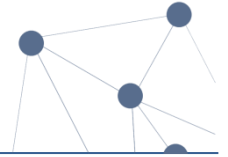
- 노드 (node)
 - 데이터 필드(data field)
 - 하나 이상의 링크 필드(link field)



- 시작/꼬리 노드, 헤드 포인터 (head pointer)



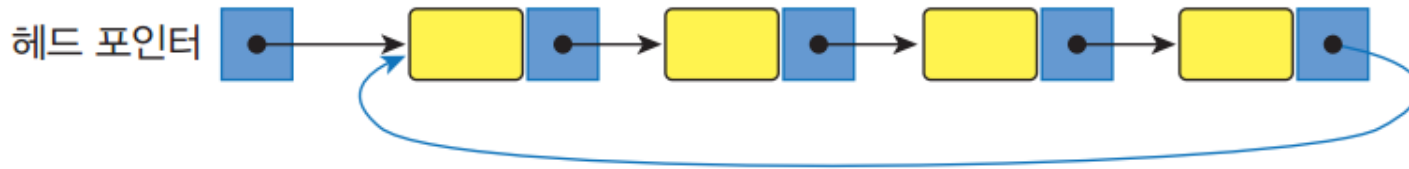
연결 리스트의 종류



- 단순 연결 리스트(singly linked list) *기관차 구조 또는 보물찾기 프로세스*



- 원형 연결 리스트(circular linked list) *음악 플레이 리스트 반복*

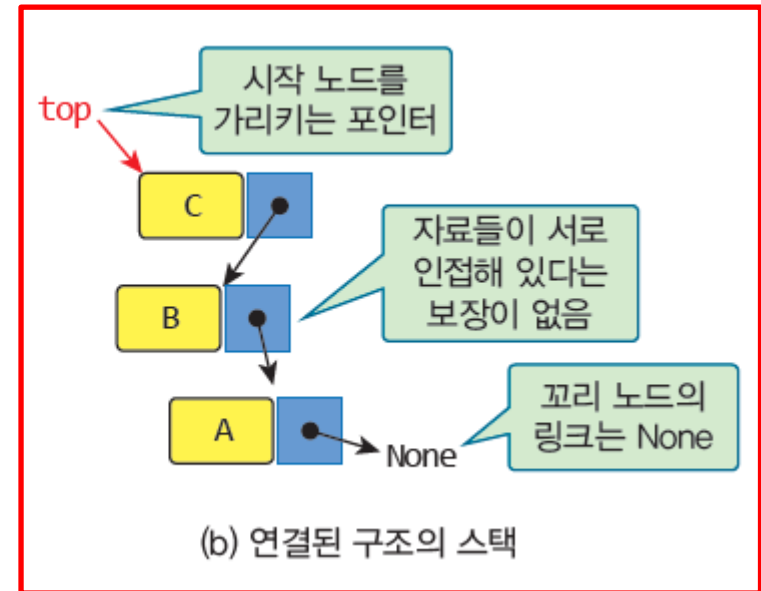
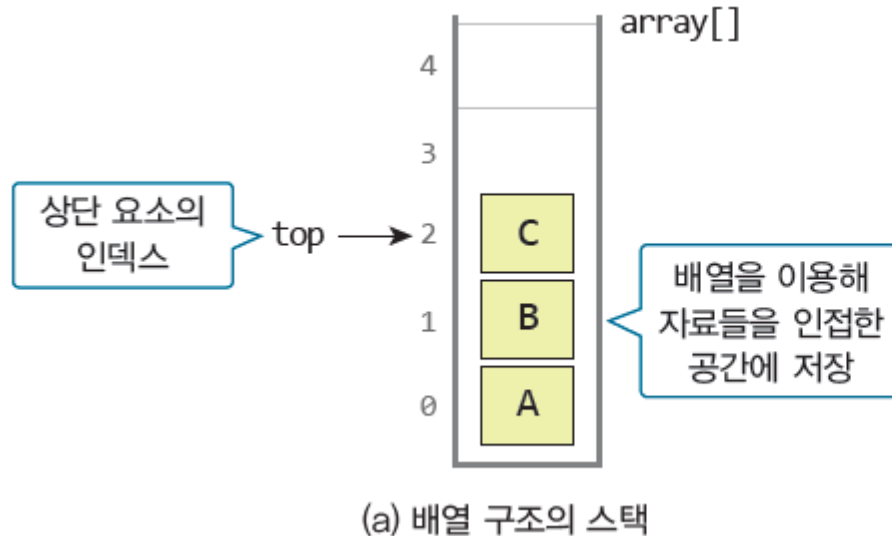


- 이중 연결 리스트(doubly linked list) *웹 브라우저 뒤로/앞으로*



6.2 단순연결리스트 응용: 연결된 스택

- 배열 구조의 스택(4장)과 연결된 스택의 구조 비교



연결된 스택 구현



pp204, 코드 6.1~6.4
파일명 : LinkedStack.py

```
1  # 코드 6.1: 단순연결노드 클래스
2  class Node:                                1) 노드 클래스
3      def __init__(self, elem, link=None):
4          self.data = elem
5          self.link = link
6
7  # 코드 6.2: 연결된 스택 클래스
8  class LinkedStack:                          2) LinkedStack 클래스
9      def __init__(self):
10         self.top = None
11
12     def isEmpty(self):
13         return self.top == None
14
15     def push(self, item):                    3) 삽입 연산
16         self.top = Node(item, self.top)
17
18     def peek(self):
19         if not self.isEmpty():
20             return self.top.data
21
22     def pop(self):                          4) 삭제 연산
23         if not self.isEmpty():
24             data = self.top.data
25             self.top = self.top.link
26             return data
```

```
28
29     # 코드 6.3: 연결된 스택의 전체 요소의 수 계산
30     def size(self):
31         node = self.top
32         count = 0
33         while not node == None:
34             node = node.link
35             count += 1
36
```

```
37     # 코드 6.4: 문자열 변환을 위한 str 연산자 중복
38     def __str__(self):
39         arr = []
40         node = self.top
41         while not node == None:
42             arr.append(node.data)
43             node = node.link
44         return str(arr)
```

연결된 스택 실행



이전 파일 이어서 복붙

```
if __name__ == "__main__":
    s = LinkedStack()          # 스택 객체를 생성

    print("스택: ", s)
    msg = input("문자열 입력: ") # 문자열을 입력받음
    for c in msg:               # 문자열의 각 문자 c에 대해
        s.push(c)              # c를 스택에 삽입

    print("스택: ", s)

    print("문자열 출력: ", end="")
    while not s.isEmpty():     # 스택이 공백상태가 아니라면
        print(s.pop(), end="") # 하나의 요소를 꺼내서 출력
    print()
    print("스택: ", s)
```

실행 결과

```
스택: []
문자열 입력: my name is jay
스택: ['y', 'a', 'j', ' ', 's', 'i', ' ', 'e', 'm', 'a', 'n', ' ', 'y', 'm']
문자열 출력: yaj si eman ym
스택: []
```

1) Node 클래스



- 단순연결구조의 노드 클래스

```
class Node:
    def __init__(self, elem, link=None):
        self.data = elem
        self.link = link
```

단순연결리스트를 위한 노드 클래스
생성자. 디폴트 인수 사용
데이터 멤버 생성 및 초기화
링크 생성 및 초기화

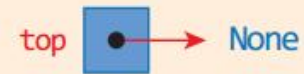
2) LinkedStack 클래스



- 클래스 정의와 생성자

```
class LinkedStack :
```

```
    def __init__( self ): # 생성자  
        self.top = None # top 생성 및 초기화
```



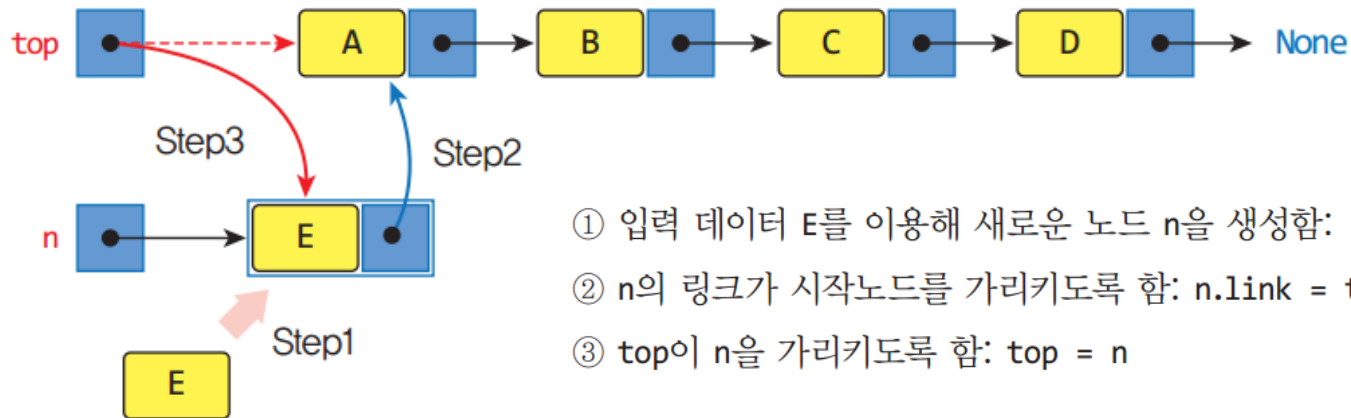
```
    def isEmpty( self ): return self.top == None # 공백상태 검사  
    def clear( self ): self.top = None # 스택 초기화
```

– 연결된 구조에서 포화 상태는 의미 없음 → 항상 False

3) 삽입 연산



- 맨 앞에 새로운 노드를 추가



- ① 입력 데이터 E를 이용해 새로운 노드 n을 생성함: $n = \text{Node}(E)$
- ② n의 링크가 시작노드를 가리키도록 함: $n.\text{link} = \text{top}$
- ③ top이 n을 가리키도록 함: $\text{top} = n$

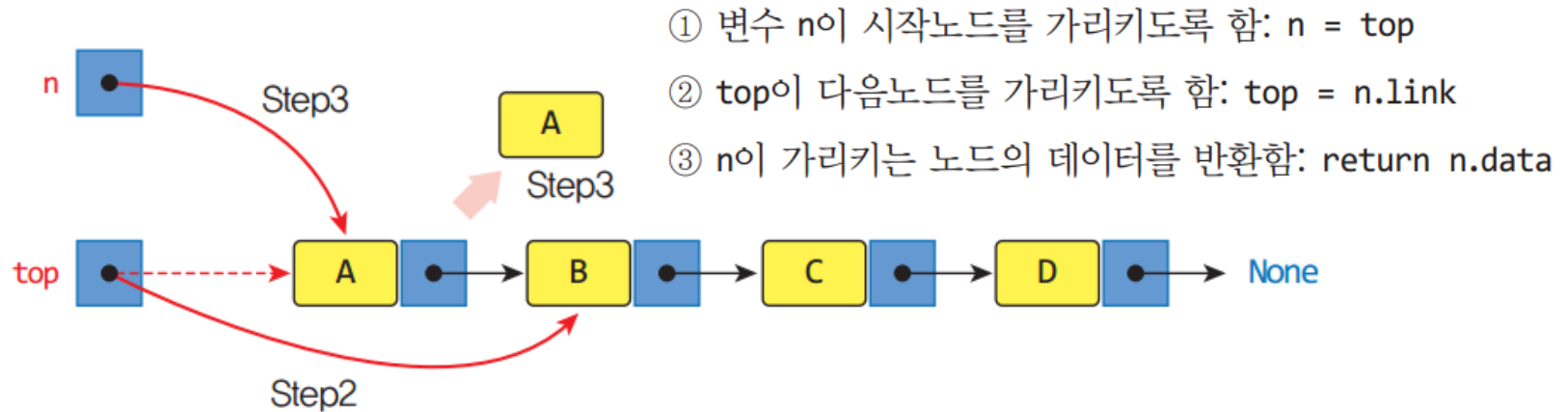
```
def push( self, item ):  
    n = Node(item, self.top)  
    self.top = n
```

```
# 연결된 스택의 삽입연산  
# Step1 + Step2  
# Step3
```

4) 삭제 연산



- 시작 노드의 데이터를 반환



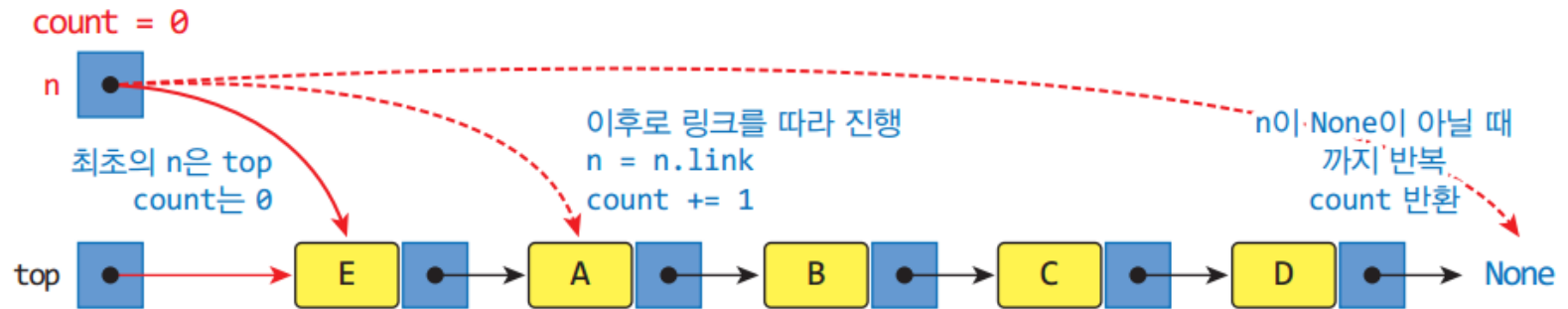
```
def pop( self ):                                # 연결된 스택의 삭제연산
    if not self.isEmpty():                       # 공백이 아니면
        n = self.top                             # Step1
        self.top = self.n.link                  # Step2
        return n.data                           # Step3
```

- 파이썬에서는 메모리 해제를 신경 쓸 필요 없음!

기타 연산들



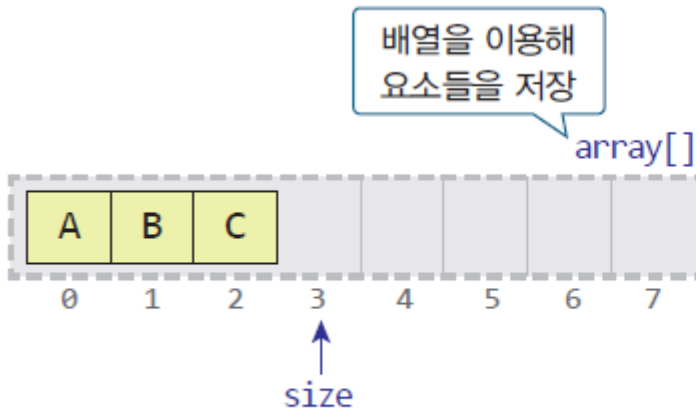
- `peek()` : 시작 노드의 데이터를 반환
- 요소의 수를 구하는 `size()`
 - 코드 6.3



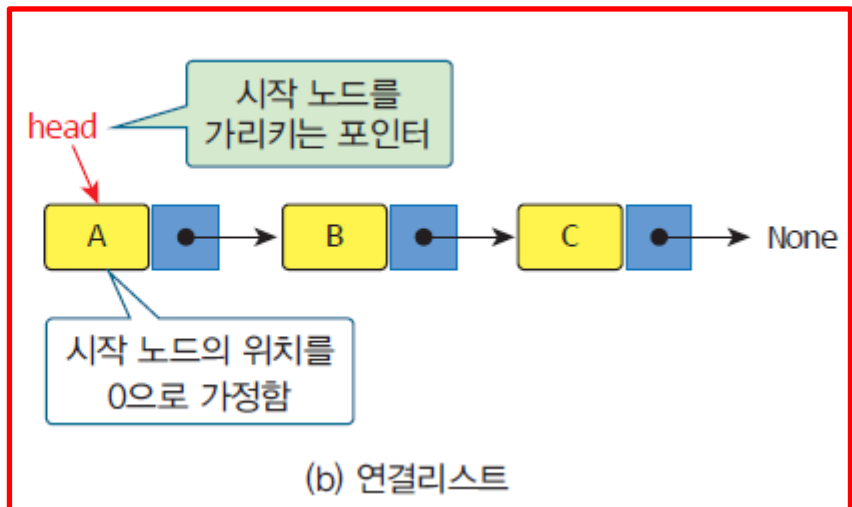
- 노드의 수를 저장하는 `count`를 `LinkedList`의 멤버 변수로 추가한다면?
- 문자열 변환을 위한 str연산자 중복
 - 코드 6.4

6.3 단순연결리스트 응용: 연결 리스트

- 배열을 이용한 리스트(3장)와 연결리스트의 구조 비교



(a) 배열 구조의 리스트



(b) 연결리스트

연결된 리스트 구현

pp212, 코드 6.5
파일명 : LinkedList.py



```
2 class Node:
3     def __init__(self, elem, next=None):
4         self.data = elem
5         self.link = next
6
7 # 코드 6.5: 연결리스트 클래스
8 class LinkedList:                                1) LinkedList 클래스
9     # 리스트의 데이터: 생성자에서 정의 및 초기화
10    def __init__( self ):
11        self.head = None
12
13    # 리스트의 연산: 클래스의 메소드
14    def isEmpty( self ): return self.head == None
15    def isFull( self ): return False
16
17    def getNode(self, pos) :                        2) 노드/항목 참조
18        if pos < 0 : return None
19        node = self.head
20        while pos > 0 and node != None :
21            node = node.link
22            pos -= 1
23        return node
24
25    def getEntry(self, pos) :
26        node = self.getNode(pos)
27        if node == None : return None
28        else : return node.data
29
30    def insert(self, pos, elem) :                    3) 삽입 연산
31        before = self.getNode(pos-1)
32        if before == None : # 맨 앞에 삽입함
33            self.head = Node(elem, self.head)
34        else :
35            node = Node(elem, before.link)
36            before.link = node
```

```
38 def delete(self, pos) :                            4) 삭제 연산
39     before = self.getNode(pos-1)
40     if before == None : # 맨 앞 노드를 삭제
41         if self.head is not None :
42             self.head = self.head.link
43     elif before.link != None :
44         before.link = before.link.link
45
46 def size( self ) :
47     node = self.head
48     count = 0
49     while node is not None :
50         node = node.link
51         count += 1
52     return count
53
54 def __str__( self ) :
55     arr = []
56     node = self.head
57     while node is not None :
58         arr.append(node.data)
59         node = node.link
60     return str(arr)
61
62 def replace(self, pos, elem) :
63     node = self.getNode(pos)
64     if node != None : node.data = elem
65
66 def find(self, val) :
67     node = self.head
68     while node is not None:
69         if node.data == val : return node
70         node = node.link
71     return node
```

연결된 리스트 실행



이전 파일 이어서 복붙

```
if __name__ == "__main__":
    L = LinkedList()

    print("최초 ", L)
    L.insert(0, 10)
    L.insert(0, 20)
    L.insert(1, 30)
    L.insert(3, 40)
    L.insert(2, 50)
    print("삽입x5 ", L)
    L.delete(2)
    print("삭제(2)", L)
    L.delete(3)
    print("삭제(3)", L)
    L.delete(0)
    print("삭제(0)", L)
```

실행 결과

```
최초      []
삽입x5     [20, 30, 50, 10, 40]
삭제(2)    [20, 30, 10, 40]
삭제(3)    [20, 30, 10]
삭제(0)    [30, 10]
```

1) LinkedList 클래스



- 클래스 정의와 생성자

```
class LinkedList:
```

```
    def __init__( self ):
```

```
        self.head = None
```

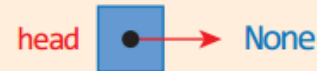
```
    def isEmpty( self ): return self.head == None
```

```
    def clear( self ) : self.head = None
```

```
    def size( self ) : ...
```

```
    def display(self, msg):...
```

연결된 리스트 클래스



공백상태 검사

리스트 초기화

self.top->self.head로 수정. 코드 동일

self.top->self.head로 수정. 코드 동일

- isEmpty(), isFull(), size() 등은 연결된 스택에서와 유사함
 - 시작 노드를 가리키는 top을 head로 바꾸기만 하면 됨

2) 노드 참조, 항목 참조

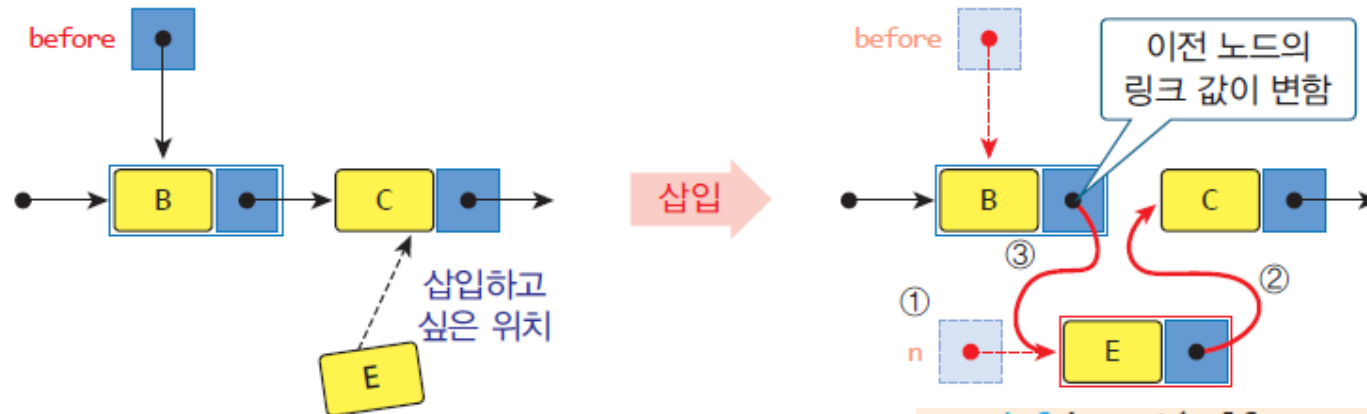
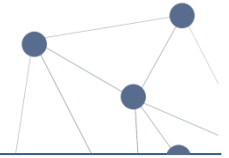


- pos번째 항목 참조를 위해서는 먼저 노드를 찾아야 함
 - getNode(pos) 메소드 추가
 - getEntry(pos)에서는 먼저 getNode(pos)를 호출해야 함

```
def getNode(self, pos) :  
    if pos < 0 : return None  
    node = self.head;  
    while pos > 0 and node != None :  
        node = node.link  
        pos -= 1  
    return node
```

```
def getEntry(self, pos) :  
    node = self.getNode(pos)  
    if node == None : return None  
    else : return node.data
```

3) 삽입 연산: insert(pos, E)

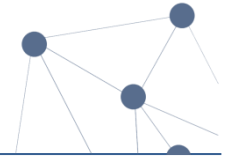


- ① E를 이용해 새로운 노드 n을 만들: `n = Node(E, None)`
- ② n의 다음 노드가 before의 다음 노드가 됨: `n.link = before.link`
- ③ before의 다음 노드가 n이 됨: `before.link = n`

- 복잡도: $O(n)$
- 만약 before를 알고 있다면?
 - $O(1)$

```
def insert(self, pos, elem) :  
    before = self.getNode(pos-1)  
    if before == None :  
        self.head = Node(elem, self.head)  
    else :  
        node = Node(elem, before.link)  
        before.link = node
```

4) 삭제 연산: delete(pos)



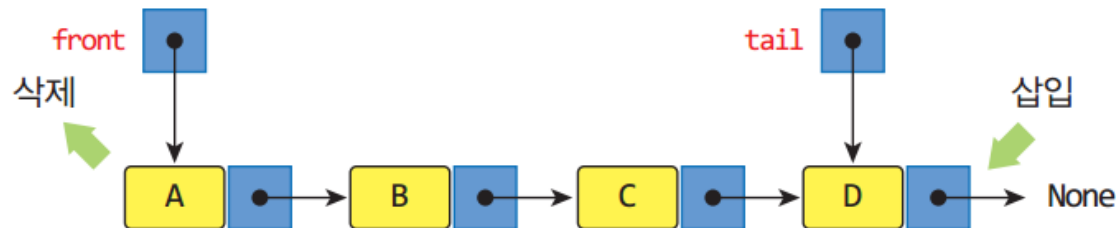
① before의 link가 삭제할 노드의 다음 노드를 가리키도록 함 : `before.link = before.link.link`

```
def delete(self, pos) :  
    before = self.getNode(pos-1)  
    if before == None :  
        if self.head is not None :  
            self.head = self.head.link  
    elif before.link != None :  
        before.link = before.link.link
```

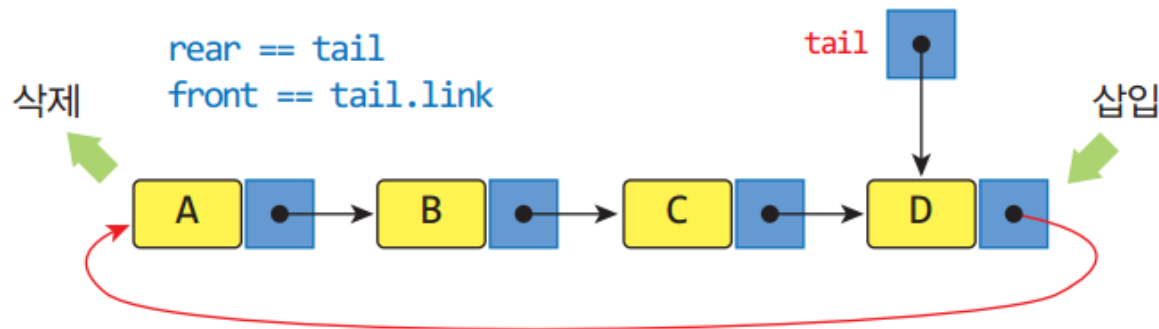
- 복잡도: $O(n)$
- 만약 before를 알고 있다면?
 - $O(1)$

6.4 원형 연결리스트의 응용: 연결된 큐

- 단순 연결 구조의 큐



- 원형 연결 구조의 큐
 - 머리 노드와 꼬리 노드를 모두 저장하고 있을 필요가 없음
 - tail만 저장해도 rear 와 front에 바로 접근할 수 있음.



연결된 큐(원형) 구현

pp217, 코드 6.6
파일명 : LinkedQueue.py



```
2 class Node:
3     def __init__(self, elem, next=None):
4         self.data = elem
5         self.link = next
6
7 # 코드 6.6: 원형으로 연결된 큐 클래스
8 class LinkedQueue:                                1) LinkedQueue 클래스
9     def __init__( self ):
10         self.tail = None
11
12     def isEmpty( self ): return self.tail == None
13     def isFull( self ): return False
14
15     def enqueue( self, item ):                      2) 삽입 연산
16         node = Node(item, None)
17         if self.isEmpty() :
18             self.tail = node
19             node.link = node
20         else :
21             node.link = self.tail.link
22             self.tail.link = node
23             self.tail = node
24
25     def dequeue( self ):                             3) 삭제 연산
26         if not self.isEmpty():
27             data = self.tail.link.data
28             if self.tail.link == self.tail : self.tail = None
29         else:
30             self.tail.link = self.tail.link.link
31         return data
32
```

```
33
34     def peek( self ):
35         if not self.isEmpty():
36             return self.tail.link.data
37
38 # 코드 6.7: 원형으로 연결된 큐의 요소의 수 계산
39 def size( self ):
40     if self.isEmpty() : return 0
41     else :
42         count = 1
43         node = self.tail.link
44         while not node == self.tail :
45             node = node.link
46             count += 1
47         return count
48
49 # 코드 6.8: 문자열 변환을 위한 str 연산자 중복
50 def __str__( self ):
51     arr = []
52     if not self.isEmpty() :
53         node = self.tail.link
54         while not node == self.tail :
55             arr.append(node.data)
56             node = node.link
57         arr.append(node.data)
58     return str(arr)
```

연결된 큐(원형) 실행



이전 파일 이어서 복붙

```
if __name__ == "__main__":
    q = LinkedQueue()
    q.enqueue('A')
    q.enqueue('B')
    q.enqueue('C')
    q.enqueue('D')
    q.enqueue('E')
    q.enqueue('F')
    print('A B C D E F 삽입: ', q)
    print('삭제 -->', q.dequeue())
    print('삭제 -->', q.dequeue())
    print('삭제 -->', q.dequeue())
    print('    3번의 삭제: ', q)
    q.enqueue('G')
    q.enqueue('H')
    q.enqueue('I')
    print('    G H I 삽입: ', q)
```

실행 결과

```
A B C D E F 삽입: ['A', 'B', 'C', 'D', 'E', 'F']
삭제 --> A
삭제 --> B
삭제 --> C
    3번의 삭제: ['D', 'E', 'F']
    G H I 삽입: ['D', 'E', 'F', 'G', 'H', 'I']
```

1) LinkedList 클래스



- 클래스 정의와 생성자

```
class LinkedList:
```

```
    def __init__( self ):  
        self.tail = None
```

연결된 큐의 생성자. 용량을 지정할 필요 없고, 데이터 멤버로 후단 노드를 가리키는 tail을 가짐

```
    def isEmpty( self ): return self.tail == None
```

tail이 None이면 공백상태

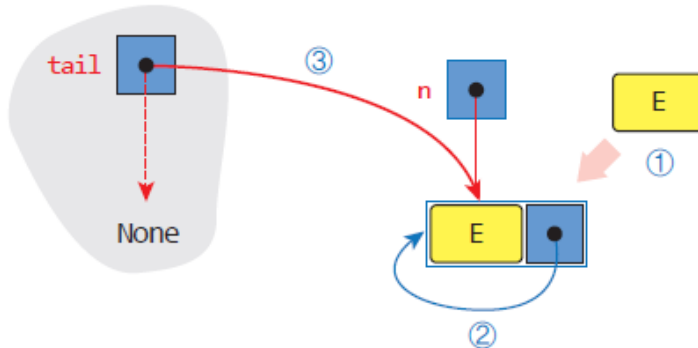
```
    def isFull( self ): return False
```

연결된 구조에서 포화상태는 의미 없음

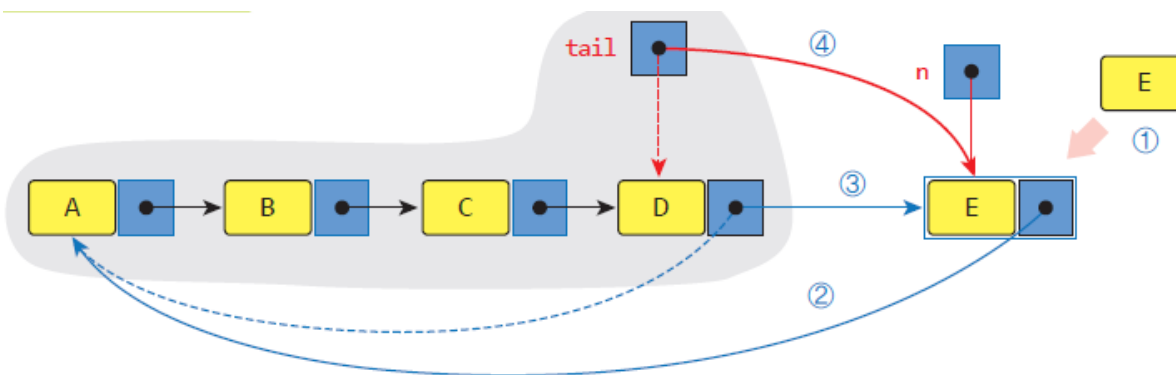
2) 삽입 연산: enqueue()



- Case1: 큐가 공백인 경우

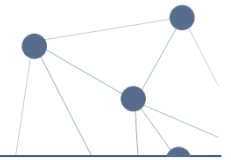


- Case2: 공백이 아닌 경우

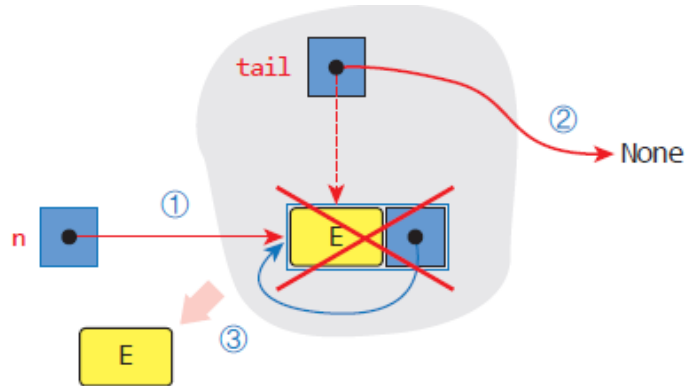


```
def enqueue( self, item ):  
    node = Node(item, None) ①  
    if self.isEmpty() :  
        self.tail = node ③  
        node.link = node ②  
    else :  
        node.link = self.tail.link ②  
        self.tail.link = node ③  
        self.tail = node ④
```

3) 삭제 연산: dequeue()

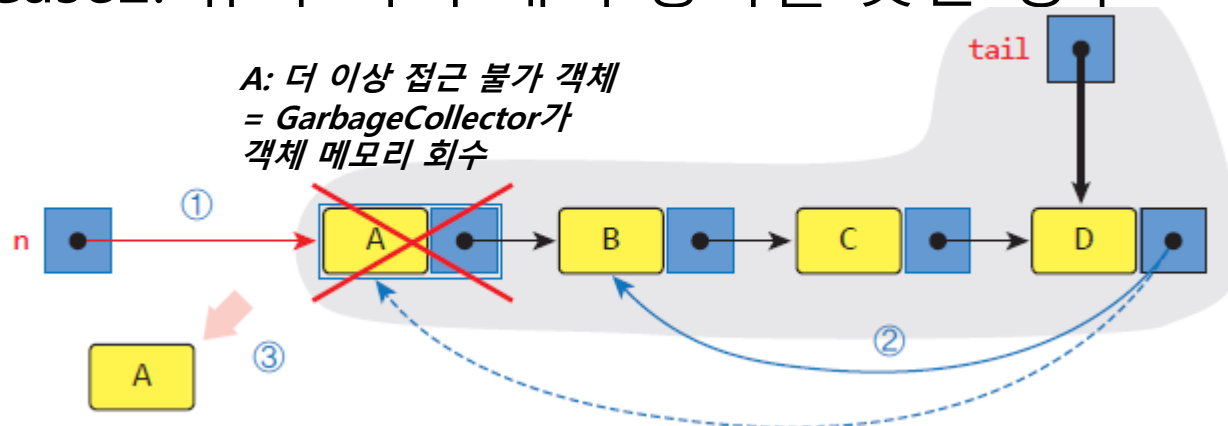


- Case1: 큐가 하나의 항목을 갖는 경우



```
def dequeue( self ):
    if not self.isEmpty():
        ① data = self.tail.link.data # 반환할 데이터
        if self.tail.link == self.tail :
            self.tail = None ②
        else:
            self.tail.link = self.tail.link.link ②
        ③ return data
```

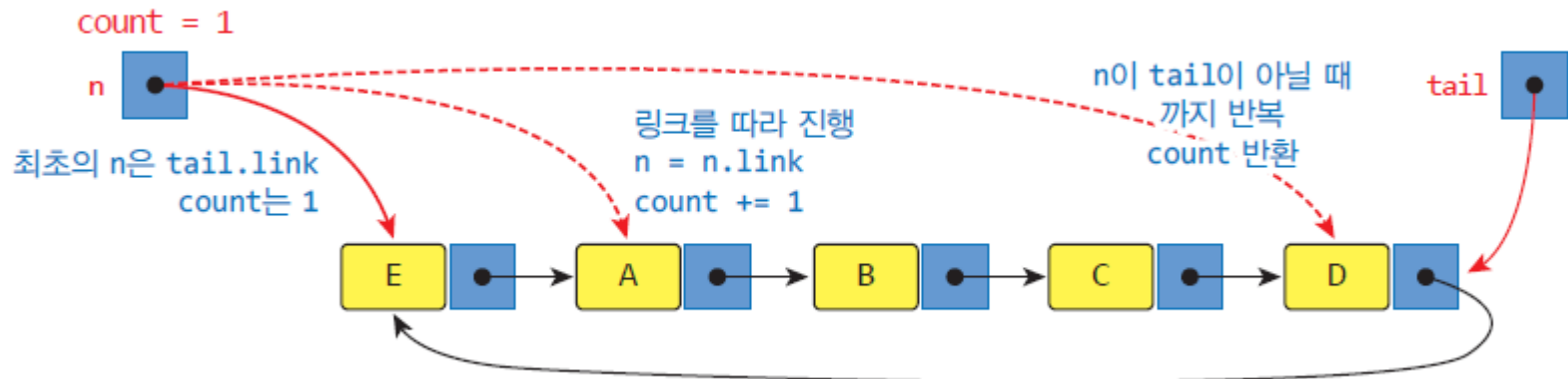
- Case2: 큐가 여러 개의 항목을 갖는 경우



기타 연산들



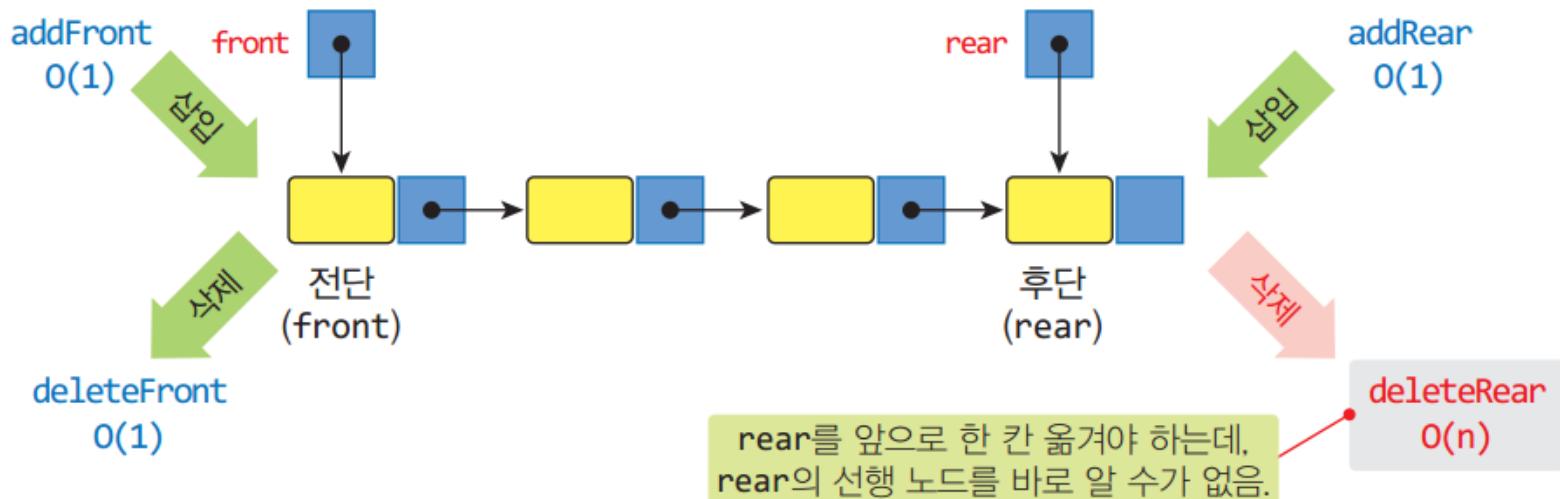
- 큐의 요소의 수 계산 size()
 - 종료 조건에 유의해야 함
 - 코드 6.7



- 문자열 변환을 위한 str 연산자 중복
 - 코드 6.8

6.5 이중연결리스트의 응용: 연결된 덩어리

- 단순 연결 구조로 구현한 덩어리의 문제점?

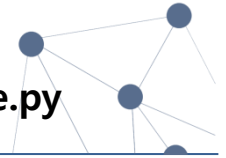


- 해결 방안은?
 - 이중연결리스트 사용

연결된 덱 구현

pp224, 코드 6.10

파일명 : DoublyLinkedDeque.py



```
2 # 코드 6.9: 이중연결구조의 노드 클래스
3 class DNode:                                1) 노드 클래스
4     def __init__(self, elem, prev, next):
5         self.data = elem
6         self.prev = prev
7         self.next = next
8
9 # 코드 6.10: 이중연결구조로 구현한 덱
10 class DoublyLinkedDeque:                    2) 덱 클래스
11     def __init__(self):
12         self.front = None
13         self.rear = None
14
15     def isEmpty(self): return self.front == None
16     def isFull(self): return False
17
18     def addFront(self, item):                 3) 삽입 (전단)
19         node = DNode(item, None, self.front)
20         if self.isEmpty():
21             self.front = self.rear = node
22         else:
23             self.front.prev = node
24             self.front = node
25
26     def addRear(self, item):                  3) 삽입 (후단)
27         node = DNode(item, self.rear, None)
28         if self.isEmpty():
29             self.front = self.rear = node
30         else:
31             self.rear.next = node
32             self.rear = node
```

```
34
35     def deleteFront(self):                   4) 삭제 (전단)
36         if not self.isEmpty():
37             data = self.front.data
38             self.front = self.front.next
39             if self.front == None:
40                 self.rear = None
41             else:
42                 self.front.prev = None
43             return data
44
45     def deleteRear(self):                    4) 삭제 (후단)
46         if not self.isEmpty():
47             data = self.rear.data
48             self.rear = self.rear.prev
49             if self.rear == None:
50                 self.front = None
51             else:
52                 self.rear.next = None
53             return data
54
55     def __str__(self):
56         arr = []
57         node = self.front
58         while not node == None:
59             arr.append(node.data)
60             node = node.next
61         return str(arr)
```

연결된 덱 실행



이전 파일 이어서 복붙

```
if __name__ == "__main__":
    dq = DoublyLinkedDeque()

    for i in range(9):
        if i%2==0 : dq.addRear(i)
        else : dq.addFront(i)
    print("홀수->전단, 짝수->후단:", dq)

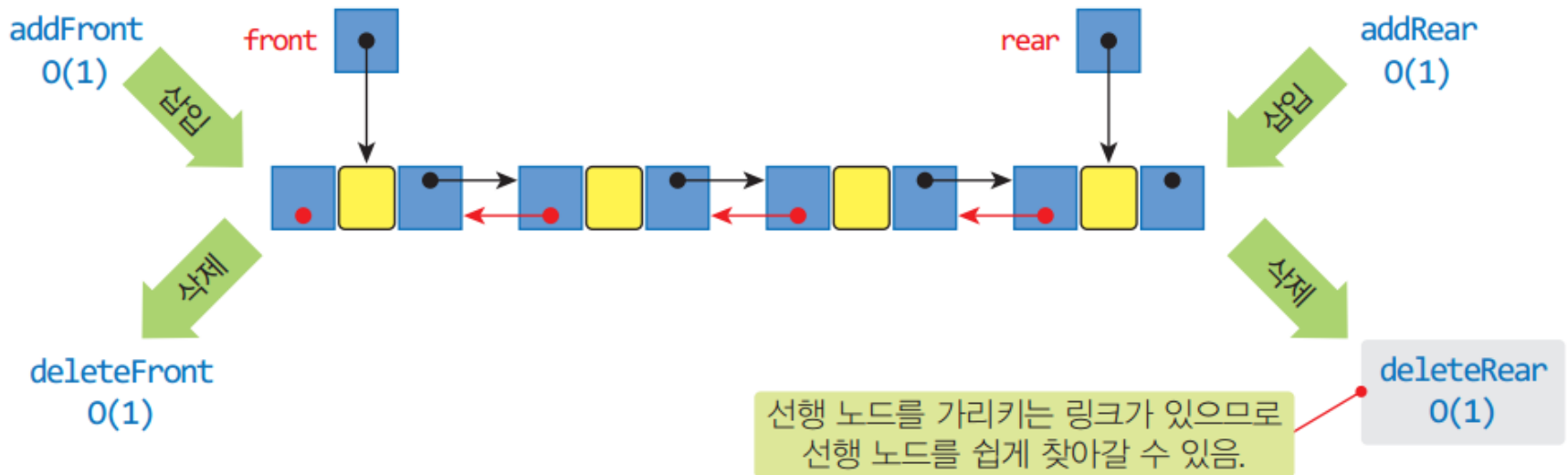
    for i in range(2): dq.deleteFront()
    for i in range(3): dq.deleteRear()
    print(" 전단삭제x2 후단삭제x3:", dq)

    for i in range(9,14): dq.addFront(i)
    print(" 전단삽입 9,10,...13:", dq)
```

실행 결과

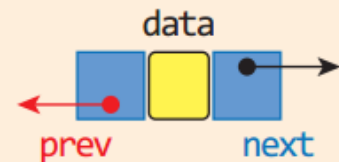
```
홀수->전단, 짝수->후단: [7, 5, 3, 1, 0, 2, 4, 6, 8]
전단삭제x2 후단삭제x3: [3, 1, 0, 2]
전단삽입 9,10,...13: [13, 12, 11, 10, 9, 3, 1, 0, 2]
```

이중 연결 구조로 구현한 덱



- 이중 연결 구조를 위한 노드 클래스

```
class DNode:                                     # 이중연결리스트를 위한 노드
    def __init__(self, elem, prev = None, next = None):
        self.data = elem
        self.prev = prev
        self.next = next
```



DoublyLinkedList 클래스

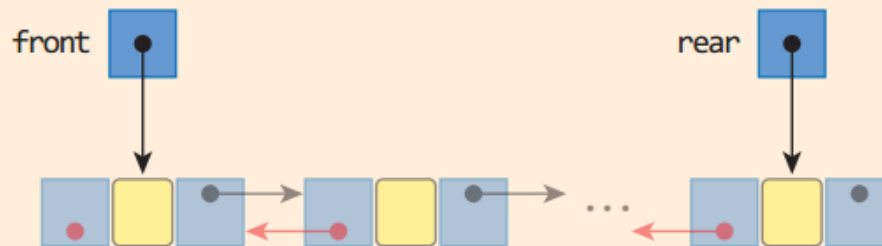


```
class DoublyLinkedList:
```

```
    def __init__( self ):
```

```
        self.front = None
```

```
        self.rear = None
```



```
    def isEmpty( self ): return self.front == None
```

공백상태 검사

```
    def clear( self ): self.front = self.rear = None
```

초기화

```
    def size( self ) : ...
```

self.top->self.front로 수정. 코드 동일

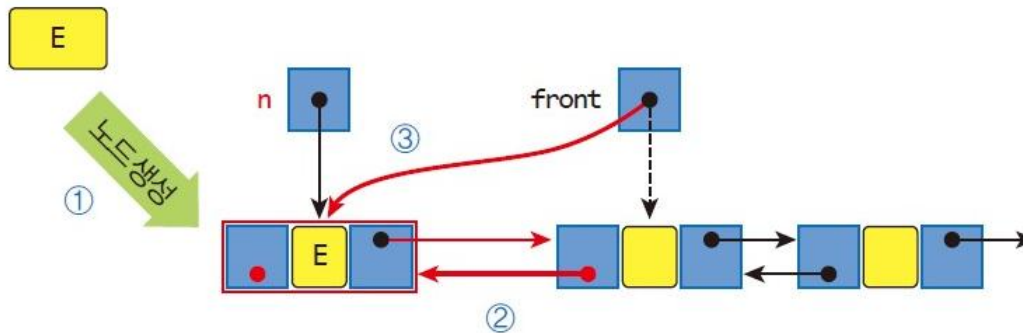
```
    def display(self, msg):...
```

self.top->self.front로 수정. 코드 동일

삽입 연산



- 전단 삽입 addFront()



[그림 6.22] 연결된 덱의 전단 삽입 과정

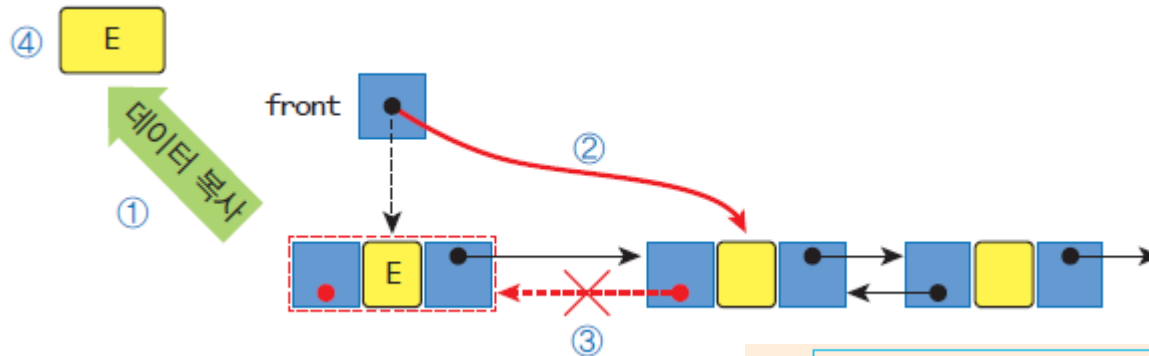
```
def addFront( self, item ):  
    node = DNode(item, None, self.front)      # ①  
    if( self.isEmpty()):  
        self.front = self.rear = node  
    else :  
        self.front.prev = node                # ②  
        self.front = node                     # ③
```

- 후단 삽입 addRear()

삭제 연산



- 전단 삭제 deleteFront()



```
def deleteFront( self ):
    if not self.isEmpty():
        data = self.front.data           # ①
        self.front = self.front.next    # ②
        if self.front==None :
            self.rear = None
        else:
            self.front.prev = None      # ③
    return data                          # ④
```

- 후단 삭제 deleteRear()

Summary



연결된 구조

- 배열 구조와 달리, 흩어진 데이터를 링크를 통해 관리(삽입/삭제 용이): 메모리 낭비x 용량고정x
- 특정 클래스 내에서 생성되는 노드 객체들로 구성
- 클래스의 head(손) -> 노드(손가락)을 사용 -> 링크(가리키는 곳)를 참조

단순 연결
(배열 리스트)
기관차/보물찾기

원형 연결
(원형 큐)
음악 재생 (목록 반복)

이중 연결
(덱)
웹브라우저: 앞으로/뒤로

삽입 순서

새로운 노드(데이터) 객체 생성
↓
기존 (양)끝 데이터의 링크
-> 새 노드 가리키도록
↓
(큐/덱의 경우)
기존 head(front/rear)가 가리키는곳
-> 새 노드(이제는 끝 데이터) 가리키도록

삭제 순서

삭제할 데이터 복사 (마지막에 return)
↓
(리스트) 삭제대상 앞 노드 링크 -> 삭제 대상 뒤 노드
(큐/덱) 끝 데이터 링크 -> None 으로
↓
(큐/덱의 경우)
기존 head(front/rear)가 가리키는곳
-> 새 노드(이제는 끝 데이터) 가리키도록



감사합니다!